B. P Poddar Institute of Management & Technology

INSIDER THREAT DETECTION SIMULATOR

Mini Project Report

Department: IT - B
Year: 4th
Semister: 7th
Session: 2026-27
Submission date: 02/11/2026

Submitted by:

Name	Rol No.
Pallavi Kumari	11500223107
Priyam Kumar	11500223108
Rocky Sarkar	11500223109
Sabir Alam	11500223110

Abstract

Insider threats present one of the most severe challenges in contemporary cybersecurity, as malicious or negligent actors operate with legitimate credentials and internal permissions, effectively bypassing conventional perimeter defenses. This mini project presents a real-time, rule-based User Behavior Analytics (UBA) detection system that demonstrates how suspicious insider activity can be detected through behavioral indicators. The application uses synthetic event records and normalized activity logs rather than destructive access. A Java Spring Boot backend analyzes event patterns and calculates a risk score, while an interactive web frontend displays the score, risk level, reasons, metric cards and event log.

1. Introduction

Traditional detection based only on known malware signatures or external perimeter firewalls may not identify every new, internal or modified threat. Behavior-based detection instead looks for unusual actions, such as rapid off-hours access, repeated failed authentication, sudden privilege self-promotions, and unusually high-volume confidential downloads. This project demonstrates the basic concept using controlled sample data and synthetic scenarios.

2. Problem Statement

The aim is to build a small educational application that can distinguish ordinary user activity from suspicious insider-threat-like patterns and explain why a scenario was flagged. The simulator must remain safe, provide clear explainability, and process high-volume events with sub-second response latency.

3. Objectives

- Demonstrate the concept of behavior-based insider threat detection.
- Create normal, suspicious and exfiltration-like synthetic scenarios.
- Analyze event counts and volume metrics using transparent heuristic rules.
- Display a risk score, risk category and detection reasons.
- Provide a simple web interface connected to a high-performance backend.

4. Tools and Technologies

Frontend: HTML, CSS, JavaScript, and React 19. Backend: Java Spring Boot 3.3.4. Data: predefined synthetic JSON-style event records and CSV audit logs. Browser communication: HTTP REST requests to a local API. The project can be run locally with Java 17+ and Maven, or accessed via the hosted web dashboard.

5. System Architecture

The user selects one of three scenarios in the frontend or uploads CSV logs. JavaScript sends the selected scenario to the API endpoint `/api/logs`. The backend loads the corresponding synthetic events, evaluates behavioral indicators, calculates a score from 0 to 100, assigns LOW, MEDIUM, HIGH or CRITICAL risk, and returns JSON. The frontend displays the returned result, threat indicators, and event log.

6. Methodology and Detection Logic

The detector uses simple explainable rules. It adds 25 points when an off-hours login occurs between 23:00 and 05:00; 25 points for at least 4 failed login attempts in a 10-minute sliding window; 20 points for at least 10 file downloads; 25 points when total transferred bytes exceed 150 MB; and 35 points when an unauthorized role change to ROLE_ADMIN is attempted.

The final score is capped at 100. Scores below 30 are LOW, 30–64 are MEDIUM, 65–84 are HIGH, and 85 or above are CRITICAL. These thresholds are illustrative and provide clear, transparent detection reasoning.

Code:

```
if off_hours_login and (hour >= 23 or hour < 5):
    score += 25
    reasons.append("Off-hours access detected outside operating schedule")

if failed_login_count >= 4:
    score += 25
    reasons.append("Repeated failed login attempts in 10-min window")

if download_count >= 10:
    score += 20
    reasons.append("High volume of file downloads detected")

if total_bytes_transferred >= 150 * 1024 * 1024:
    score += 25
    reasons.append("Rapid mass data exfiltration volume detected (>150MB)")

if role_change and new_role == "ROLE_ADMIN":
    score += 35
    reasons.append("Unauthorized administrative privilege escalation attempted")

score = min(score, 100)

if score >= 85:
    level = "CRITICAL"
    message = "Critical insider threat detected. Revoke session and escalate immediately."
elif score >= 65:
    level = "HIGH"
    message = "Insider-like behavior detected. Isolate the simulated account and investigate."
elif score >= 30:
    level = "MEDIUM"
    message = "Suspicious activity. Continue monitoring and review the event details."
else:
    level = "LOW"
    message = "No strong insider threat indicators in this synthetic event set."
```

7. Implementation

The backend exposes `/api/logs`, `/api/logs/upload`, `/api/threats` and `/api/threats/resolve`. The frontend provides three scenario buttons and an Analyze / Simulator button. It uses Fetch API to send a POST request and renders the response without reloading the page. The custom CSV upload endpoint accepts enterprise audit logs for further experimentation.

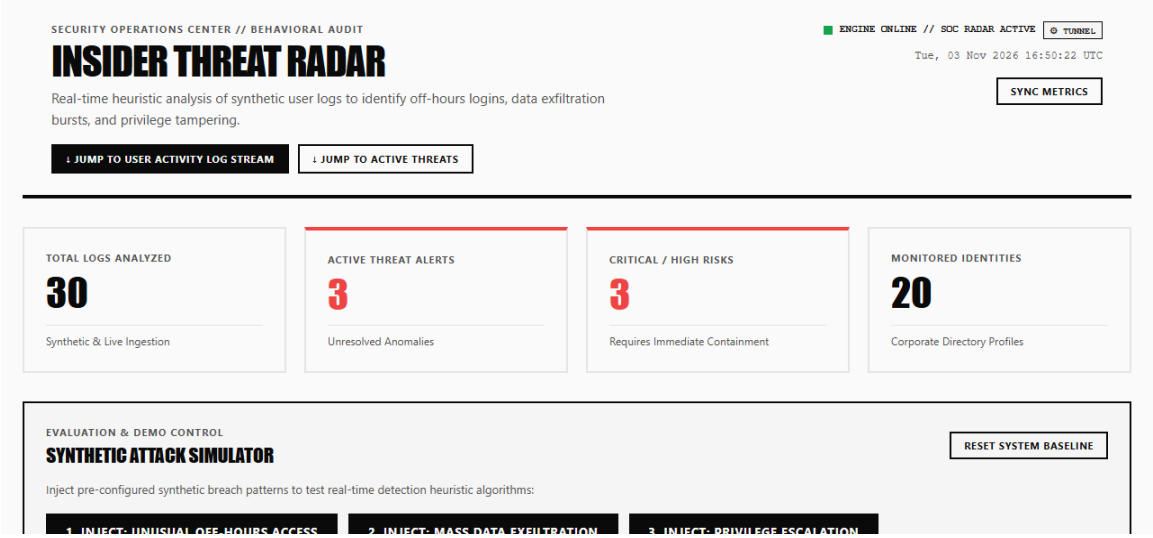
8. Testing and Expected Results

Normal scenario: low activity and routine daytime views; expected risk is LOW. Suspicious scenario: off-hours access combined with repeated downloads; expected risk is MEDIUM or HIGH depending on the combined score. Threat scenario: high-volume writes, mass file downloads, and privilege escalation; expected risk is HIGH or CRITICAL. The exact score is calculated by the backend rules. Test each scenario and compare displayed metrics.

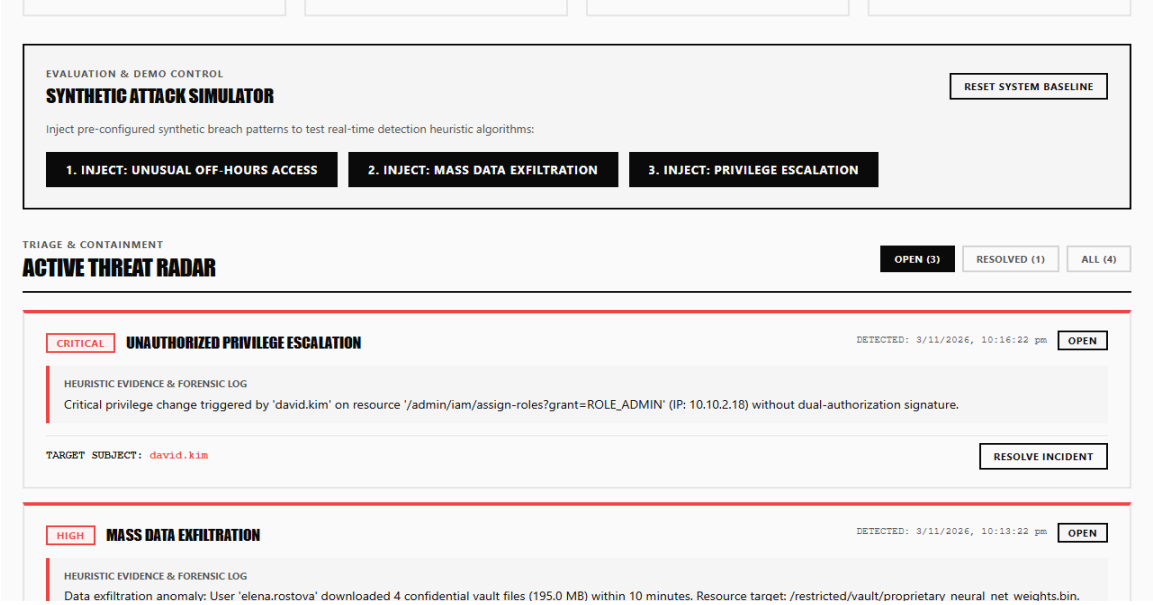
Demo & Example

Live URL: https://codewithpriyam.github.io/insider-threat-detection/

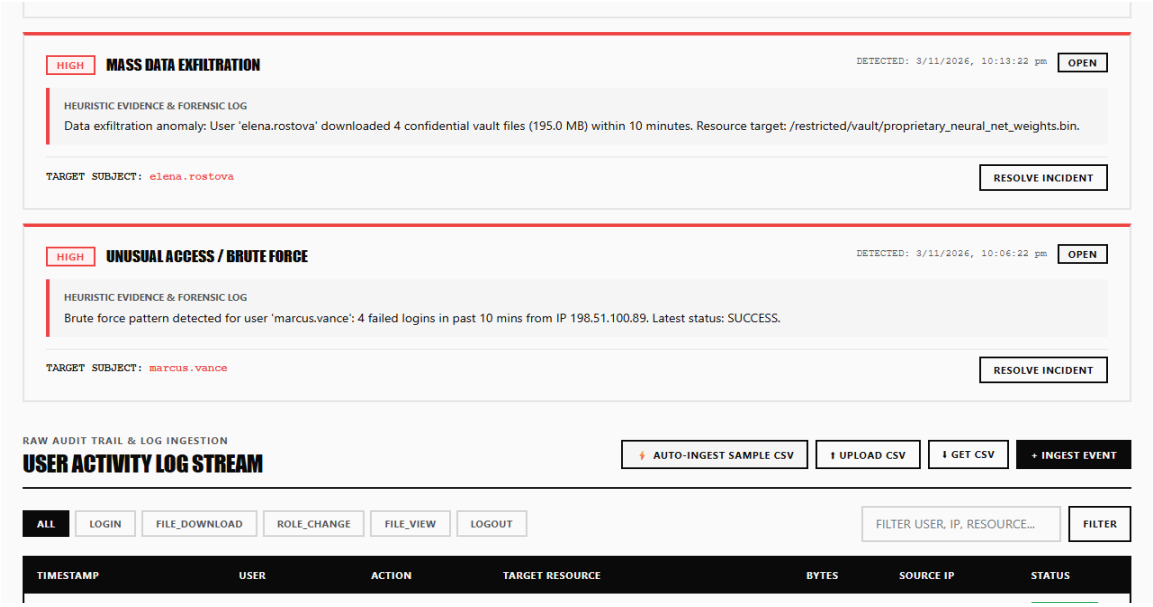
1.



2.


EVALUATION & DEMO CONTROL
SYNTHETIC ATTACK SIMULATOR
RESET SYSTEM BASELINE
Inject pre-configured synthetic breach patterns to test real-time detection heuristic algorithms:
1. INJECT: UNUSUAL OFF-HOURS ACCESS
2. INJECT: MASS DATA EXFILTRATION
3. INJECT: PRIVILEGE ESCALATION
TRIAGE & CONTAINMENT
ACTIVE THREAT RADAR
OPEN (3)
RESOLVED (1)
ALL (4)
CRITICAL
UNAUTHORIZED PRIVILEGE ESCALATION
DETECTED: 3/11/2026, 10:16:22 pm
OPEN
HEURISTIC EVIDENCE & FORENSIC LOG
Critical privilege change triggered by 'david.kim' on resource '/admin/iam/assign-roles?grant=ROLE_ADMIN' (IP: 10.10.2.18) without dual-authorization signature.
TARGET SUBJECT: david.kim
RESOLVE INCIDENT
HIGH
MASS DATA EXFILTRATION
DETECTED: 3/11/2026, 10:13:22 pm
OPEN
HEURISTIC EVIDENCE & FORENSIC LOG
Data exfiltration anomaly: User 'elena.rostova' downloaded 4 confidential vault files (195.0 MB) within 10 minutes. Resource target: /restricted/vault/proprietary_neural_net_weights.bin.

3.


HIGH
MASS DATA EXFILTRATION
DETECTED: 3/11/2026, 10:13:22 pm
OPEN
HEURISTIC EVIDENCE & FORENSIC LOG
Data exfiltration anomaly: User 'elena.rostova' downloaded 4 confidential vault files (195.0 MB) within 10 minutes. Resource target: /restricted/vault/proprietary_neural_net_weights.bin.
TARGET SUBJECT: elena.rostova
RESOLVE INCIDENT
HIGH
UNUSUAL ACCESS / BRUTE FORCE
DETECTED: 3/11/2026, 10:06:22 pm
OPEN
HEURISTIC EVIDENCE & FORENSIC LOG
Brute force pattern detected for user 'marcus.vance': 4 failed logins in past 10 mins from IP 198.51.100.89. Latest status: SUCCESS.
TARGET SUBJECT: marcus.vance
RESOLVE INCIDENT
RAW AUDIT TRAIL & LOG INGESTION
USER ACTIVITY LOG STREAM
AUTO-INGEST SAMPLE CSV
UPLOAD CSV
GET CSV
+ INGEST EVENT
ALL
LOGIN
FILE_DOWNLOAD
ROLE_CHANGE
FILE_VIEW
LOGOUT
FILTER USER, IP, RESOURCE...
FILTER
TIMESTAMP
USER
ACTION
TARGET RESOURCE
BYTES
SOURCE IP
STATUS

9. Advantages

- Simple and easy to demonstrate.
- Explainable rules make the result understandable.
- Separates frontend and backend responsibilities.
- Uses synthetic data, so it is safe for classroom use.
- Can be extended with real endpoint telemetry in a controlled research environment.

10. Limitations and Future Scope

This is a rule-based prototype and does not inspect real OS kernel system calls. It may produce false positives and false negatives. Future work could include collecting benign activity baselines, adding machine-learning classifiers on vetted datasets, adding alert history, and evaluating precision, recall and F1-score. Any real-world monitoring should be authorized, privacy-aware and tested in an isolated environment.

11. Conclusion

The project demonstrates the fundamental idea that insider-threat-like behavior can be identified through combinations of unusual user file operations and access events. A lightweight Spring Boot backend evaluates synthetic events and a web dashboard communicates the result clearly. The simulator is intended for learning and demonstration, not for production security protection.

References

1. CISA, Insider Threat Mitigation Guidance: https://www.cisa.gov/insider-threat-mitigation
2. NIST Cybersecurity Framework: https://www.nist.gov/cyberframework
3. Spring Boot documentation: https://spring.io/projects/spring-boot